mRNA Design and Optimization with Deep Knowledge-Infused Approach

Zheng Gong[a,1], Ziyi Jiang[a,1], Weihao Gao[a], Yuanyuan Wang[a], Zhining Cai[a], Zhuo Deng[a] and Lan Ma[a,b,*]

[a]Tsinghua Shenzhen International Graduate School, Shenzhen, 518055, Guangdong, China
[b]Shenzhen Bay Laboratory, Shenzhen, 518107, Guangdong, China

arXiv:2505.23862v2 [q-bio.QM] 30 Aug 2026

ARTICLE INFO

Keywords:
Knowledge-infused Learning
mRNA Design

ABSTRACT

The mRNA optimization is essential for mRNA vaccines, therapies, and industrial protein production. Based on current explorations, an ideal optimization approach should simultaneously (i) prevent unintended amino-acid changes, (ii) optimize multiple, biologically relevant objectives, and (iii) retain computational efficiency. However, existing methods are forced to trade off between these perspectives, forming an "impossible triangle." We present RNop, a knowledge-infused Transformer that integrates mechanism-aligned losses to address this problem. By encoding biological prior knowledge in losses, RNop makes knowledge infusion explicit and controllable across optimization focus. Trained on over 6 million sequences, *in silico* analyses show RNop resolves the "impossible triangle" of mRNA optimization with absolute sequence fidelity, significantly improved biological metrics, and high throughput. In *in vitro* validation, it can deliver up to 2.28-fold expression gain. Ablation studies reveal how each prior contributes to targeted improvements, yielding mechanism-level interpretability. RNop represents a shift in mRNA optimization methodology: by infusing explicit and interpretable knowledge, the "black-box" mRNA design can be transformed into a predictable, explainable engineering problem. RNop is designed as an extensible platform: additional biological priors can be incorporated as modular, mechanism-aligned loss functions, enabling future development and adaptation to related sequence design problems.

1. Introduction

The design and optimization of messenger RNA (mRNA) sequences represent a critical consideration in modern biological and biomedical research. As the mRNA sequence dictates translation efficiency and subsequent protein yield, challenges in achieving robust heterologous expression of target genes have driven the development of mRNA optimization methods. This optimization is now indispensable for a wide range of biotechnological goals, encompassing large-scale protein production, gene and cell therapies, synthetic biology circuits, and new medicine strategies. The performance of mRNA molecules as therapeutic agents or research tools hinges on their sequence characteristics, directly impacting efficacy and production scalability. Advancements in mRNA design and optimization also contribute to recombinant protein manufacturing and drug discovery pipelines [15]. Given the expanding therapeutic and industrial applications of mRNA [32], refining optimization techniques remains an area of intense investigation.

Current mRNA optimization algorithms can be assessed based on several key criteria as shown in Fig.1 (a): (1) **fidelity** (preventing unintended amino acid changes), (2) **the scope of optimization variables considered** (optimality in multiple biological metrics), and (3) **computational efficiency** (speed and scalability). However, they are forced to trade off between these essential criteria, constrained by a central challenge, which we term the "impossible triangle" of mRNA optimization. To date, no existing method can simultaneously satisfy all three objectives, creating a significant bottleneck for advanced mRNA engineering.

*Corresponding author
hudenjear@gmail.com (Z. Gong); jiang-zy20@mails.tsinghua.edu.cn (Z. Jiang); gwh20@mails.tsinghua.edu.cn (W. Gao); wang-yy24@mails.tsinghua.edu.cn (Y. Wang); caizn24@mails.tsinghua.edu.cn (Z. Cai); dengzhuo150@gmail.com (Z. Deng); malan@sz.tsinghua.edu.cn (L. Ma)
ORCID(s):
[1]These authors contributed equally to this work.

This "impossible triangle" arises from the inherent limitations of current design paradigms. Algorithmic methods like Dynamic Programming (DP) can ensure perfect sequence fidelity [41, 40, 43] and incorporate multiple optimization factors [45]. However, their computational complexity makes them slow for long sequences and incapable of high-throughput applications. Conversely, deep learning models [2, 11, 14] offer excellent computational efficiency and scalability. However, as probabilistic models, they struggle to guarantee absolute sequence fidelity, often introducing undesired protein mutations. Furthermore, typically, they learn optimization rules implicitly from natural sequence data [11, 14, 13, 44], which limits their ability to explicitly and concurrently optimize for multiple, well-studied biological metrics.

From a technical perspective, the central challenge is enforcing strict synonymous constraints within deep learning models while still supporting rich, multi-objective optimization. Conventional sequence-to-sequence or language-model-based approaches operate in codon token space and can only reduce, but not eliminate, protein-level mutations; post-hoc "repair" of mutated codons often destroys the jointly optimized distribution and offers no principled way to balance fidelity against exploration. In addition, when all biological factors are entangled in a single loss, it becomes difficult to attribute improvements to specific mechanisms or to tune their relative influence. These limitations motivate the need for a mechanism that can encode biological priors as differentiable constraints at the protein level and expose their effects in a controllable way.

The root of this problem lies in the methodology. Most deep learning approaches treat the optimization process as a black box [44]. These models learn latent knowledge from sequences, without leveraging accumulated biological knowledge. Therefore, we propose a different paradigm: we explicitly infuse established biological prior knowledge in the model training process through loss functions. This approach, knowledge-infused learning (K-iL), can leverage prior knowledge to enhance model performance and has achieved success in physics and other fields. [35, 31, 18] To the best of our knowledge, RNop is the first application of the K-iL in mRNA optimization, extending the spirit of physics-informed neural networks (PINNs) [35] to mRNA sequence design.

To be more specific, we introduce RNop, a novel deep learning framework that embodies this knowledge-infused methodology to solve the “impossible triangle”. At its core, RNop introduces a matrix-based constraint enforcement mechanism that enables strict synonymous CDS optimization in deep learning. This mechanism is instantiated on top of a Vision Transformer (ViT) backbone that treats mRNA optimization as an image-to-image translation problem. Meanwhile, RNop learns from explicit, mechanism-aligned loss functions rather than directly regressing target sequences. We developed four specialized loss functions: Gene-Protein Loss (GPLoss), CAILoss, tAILoss, and MFELoss. One of Our key technical contributions is the linear-algebraic formulation of these losses, which makes protein-gene correlation knowledge natively incorporated in deep learning. GPLoss operates directly on the codon probability space and penalizes non-synonymous probability mass, thereby enforcing protein-level fidelity during optimization. This is fundamentally different from post-processing “repair” strategies that replaces mutated codons after decoding [11], which may cancel optimization gains. Meanwhile, in RNop, the GPLoss weight provides such control: sufficiently high weights yield strict synonymy, while lower weights intentionally allow limited exploration when desired. Building on the same design philosophy, CAILoss and tAILoss incorporate codon-usage bias [39] and tRNA adaptation [36, 37] for a target species, and MFELoss incorporates secondary-structure considerations by minimizing predicted MFE, which is associated with mRNA stability and potentially longer half-lives [27, 25, 3]. Together, these losses enable one-time multi-objective optimization of full CDS sequences.

By encoding biological priors as objective-aligned losses, RNop makes knowledge infusion explicit and enables a controllable, multi-objective optimization that was previously unattainable. The brief framework of our methods is shown in Fig.1 (b), and a detailed description is given in Section. Method. We outline the optimization process in Fig.1 (c), in which codon "ACG" is selected as an example to show how the knowledge learned from four loss functions influences the output probability distribution during the inference.

Then, we trained and validated RNop models on a dataset of 6,000,000 mRNA sequences. Extensive *in silico* analysis demonstrates that RNop consistently improves all target metrics across multiple species while maintaining perfect sequence fidelity. Crucially, in *in vitro* biological validation experiments, RNop-optimized sequences achieved up to a 2.28-fold increase in protein expression compared to commercially available and highly optimized sequences, confirming the real-world efficacy of RNop. With a processing speed of up to 44 sequences per second, RNop also solves the efficiency problem, enabling high-throughput design. Through this work, we provide a high-fidelity, interpretable, and scalable solution that breaks the "impossible triangle." Beyond a single model, RNop is designed as a modular and extensible platform: new biological priors and species-specific objectives can be incorporated as plug-in mechanism-aligned loss functions, and new network structures can be applied. We hope this project can

support future development and adaptation to related sequence design problems. Our code and models are available at https://github.com/HudenJear/RPLoss.

1.1. Materials and Methods

1.1.1. Dataset

The genome data is downloaded from the NCBI database. We downloaded the gene sequences of Eukaryota and Bacteria from the NCBI Genome Database[29]. The Eukaryota and Bacteria sequences are filtered: only the complete and known mRNA (transcriptions) are included. The virus genome data is downloaded from the NCBI Virus Database[30]. The virus sequences are filtered: only the sequences that are complete and have lengths from 1 to 30000 were downloaded. All sequences were downloaded prior to May 1th 2025. In the initial stage, 21,194,000 sequences are downloaded.

We preprocess these raw sequence data into the dataset. In this step, the sequences are cropped to the UTRs and CDS sequences. The duplicated CDS sequences (mRNA) are deleted to avoid data leaking for validation and testing. For the basic RNop model, the sequences over 45 and under 3072 nt are included. For the extended RNop model with doubled and 4-fold maximum lengths, the sequences over 45 nt and under 6144/12288 nt are included.

To balance the data sources, we randomly sampled 2,000,000 sequences from each category: Eukaryota, Bacteria, and Virus. Therefore, we collected an mRNA sequence dataset containing 6,000,000 mRNA sequences for training, validation, and testing with balanced categories. Compared to 38000[14], 42266[15], or 71820[1], the amount of mRNA sequences is larger than previous codon optimization methods.

The sampled sequences are further divided into train, validation, and test sets, with partitions of 90%, 1%, and 9%, respectively. There are 5,400,000 sequences in the training data, and 60,000 and 540,000 in the validation and test data. Meanwhile, to test the homologous optimization ability, another species-specified test set of 100,000 sequences (25,000 in virus, E.coli, mouse, and human) was built, using the data not included in the sampling step.

1.1.2. Biological Experiment Setting

In vitro experiments. The HEK293T cell line used in this study was preserved in our laboratory.

mRNA transfection: HEK293T cells were cultured in DMEM medium(GIBCO, #11965092) supplemented with 10% fetal bovine serum (GIBCO, #A5670701) and 1% penicillin-streptomycin(YEASEN ,#60162ES), and maintained in a 37°C incubator with 5% carbon dioxide. mRNA was transfected into HEK293T cells using Lipofectamine™ MessengerMAX™ Transfection reagent(Invitrogen, #LMRNA015) following the manufacturer's instructions. At the appropriate time after transfection, cells and cell lysates were collected for fluorescence and protein detection.

In Vitro Protein Detection: To detect the in vitro expression of fluorescent proteins EGFP and ECFP, HEK293T cells were seeded in 24-well plates, with 1 μg of mRNA transfected into each well. At 24 h and 48 h post-transfection, the fluorescence intensity of cells in each well was measured using a fluorescence spectrophotometer. Meanwhile, cells were harvested at the same time points, lysed, and subjected to western blot analysis with EGFP(ABCAM, #ab184601) and ECFP(ABBKINE, #abt2280) antibodies. Additionally, western blot detection was performed on the optimized sequences of other target proteins using His antibodies(PROTEINTECH, #66005-1-Ig).

1.1.3. Brief look of RNop

Our RNop includes a coding method for various sequences, 4 knowledge-infused loss functions incorporating biological prior knowledge, and a new Transformer-based deep learning model, the RNop model.

The coding method is the cornerstone of our method. With this coding rule, we are able to encode multiple types of gene sequences into uniformly formatted feature maps. Then, to leverage the prior knowledge from biology, we create four loss functions to guide the deep learning model training: GPLoss, CAILoss, tAILoss, and MFELoss. Furthermore, the RNop model is designed to optimize the mRNA CDS with multiple optimization factors, high sequence fidelity, and high throughput. Through our coding method, the RNop model obtains the ability to process mRNA, DNA CDS, or protein sequences and optimize gene sequences species-specifically.

1.1.4. mRNA Optimization Model: RNop

The mRNA sequences are simpler on the one hand– they usually have less vocabulary–and complicated on the other hand–they are more strict with length and coding. Therefore, we designed a new model for mRNA optimization: the RNop model. The RNop model is a sequence-to-sequence model that consists of the Transformer and convolutional network layers and leverages the Vision Transformer (ViT) [10, 22] architecture. The choice of the ViT model is not

an arbitrary decision. We have tested a wide range of deep learning models and found that the ViT model is the only model that can fully avoid errors and utilize the feature of knowledge-infused loss functions.

The RNop model leverages the ViT architecture to accomplish the sequence-to-sequence task. Its architecture is shown in Fig.2. As discussed above, we can choose mRNA, DNA, or amino acids coding for the input. After the encoding process, the $< EOS >$ and $< PAD >$ are added to the input, and the sequences are transformed to codon possibility space matrices. We assume the input sequences I have the shape of $B \times L \times V$, where the B, L, and V represent the batch size, sequence length, and vocabulary size. To standardize the input dimension, the input I is processed by an input projection layer to adjust its shape to $X_1 \in \mathbb{R}^{B\times L\times C}$, where C represents the projection channel. The feature X_1 is reshaped to $X_2 \in \mathbb{R}^{B\times \frac{L}{C}\times C\times C}$, which is akin to an image with $\frac{L}{C}$ channels, height C and width C.

Subsequently, the image-like feature is processed by a feature embedding layer, which is a convolutional layer with a stride of 1 and a kernel size of 3. (We observed severe information loss when we applied the non-overlapping patch embedding layer.) The processed feature is $X_3 \in \mathbb{R}^{B\times F\times C\times C}$, where F is the feature embedding dimension. Then, X_3 is processed by 16 Shifted Window-based Multihead Self-Attention layers[22]. This attention mechanism ensures that the global feature and local feature can be extracted and fused. The feature size is kept the same during this process, and no downsampling or upsampling is applied. We call this component the ViT encoder. The output of the last ViT encoder layer is $O_1 \in \mathbb{R}^{B\times F\times C\times C}$. Then, the O_1 is processed by a point-wise convolutional layer, modifying its dimension from F to $\frac{L}{C}$ and further reshaped back to $O_2 \in \mathbb{R}^{B\times L\times C}$.

Consequently, we utilize a head layer to adjust the last dimension of the O_2 from C to V to give the final output $O \in \mathbb{R}^{B\times L\times V}$. Then the output can be decoded to mRNA sequences or used directly in the loss function calculation for gradient descent.

1.1.5. Data Representation: Coding Rules

We designed a coding rule to standardize mRNA and other sequences for neural networks. While some LLMs use tokenizers and K-mers[28], they can be redundant due to the limited number of codons. Therefore, we propose a new coding method that directly converts sequences into numerical matrices to avoid redundancy.

Generally, the mRNA sequences consist of four ribonucleotides: A, U, C, and G. Meanwhile, most RNA datasets (including NCBI[29] datasets) use "T" to replace "U" to keep the representation the same as the DNA sequences. Thus, following this conventional usage, we represent RNA sequences accordingly to include both mRNA and DNA CDS. With a fixed order "$TCAG$" and 3 nucleotides in one codon, we can settle the position of each codon in the matrices. This enables us to map one codon to its coding protein using linear algebra by matrix multiplication. The vocabulary of this coding rule is $4^3 = 64$, which is also the width of the transformed matrices. Since the length is the sequence length, the matrices can represent the probability space of input and output sequences. The detailed mapping method of this coding rule can be referred to in Supplementary information.

Additionally, this coding rule is compatible with uncertain codons that are denoted as “N” or “RYMKSWHBVDN.” With five alphabets representing RNA, the vocabulary size is $5^3 = 125$. Although using more alphabets (e.g., RYMKSWHBVDN) provides richer information than a singleton “N,” it would inflate the vocabulary size to $15^3 = 3375$. As they are really rare in the sequences, we decided to only include the "N" codons in the RNop model.

Furthermore, RNA sequence lengths vary a lot. Alongside codon symbols, we introduce an $< EOS >$ (End Of Sequence) symbol at the end of RNA sequences. To ensure consistent input and output lengths for neural networks, we also add $< PAD >$ (Padding) coding symbols. Working with GPloss, these tokens can avoid length change in output sequences.

1.1.6. GPLoss: Avoiding Mutation Through Coding Rule

GPLoss is built on the coding rule and is designed to enforce *sequence fidelity* at the protein level via a differentiable, matrix-based constraint. Using the codon-to-amino-acid mapping, GPLoss penalizes output probability mass assigned to codons that would translate into an amino acid different from that of the input codon at the same position. Unlike post-hoc “repair” approaches that modify decoded sequences after generation, this formulation constrains the probability space during training and optimization inference and thus preserves the joint, multi-objective behavior of the model. With a sufficiently large loss weight, this mechanism strongly suppresses non-synonymous mutations and preserves the translated protein sequence. Conversely, reducing the GPLoss weight relaxes this constraint and can allow a controlled amount of amino-acid changes. In practice, tuning the GPLoss weight provides a direct handle on the mutation rate, ranging from 0% strictly to higher values when intentionally permitted.

Worked example (standard nucleotides only)

Assume three input codons: UGC, CAU, and AAG. These codons encode the amino acids Cysteine, Histidine, and Lysine, respectively. To illustrate the GPLoss computation end-to-end, we include the synonymous codons for these amino acids and additionally Phenylalanine as an extra example and construct the input codon matrix I as follows.

$$I = \begin{array}{c} \\ input1 : AAG \\ input2 : UGC \\ input3 : CAU \end{array} \begin{array}{c} \begin{array}{cccccccc} UGU & UGC & CAU & CAC & AAA & AAG & UUU & UUC \end{array} \\ \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{array} \tag{1}$$

Next, I is multiplied by an amino-acid encoding matrix (denoted as C) that represents the mapping between codons and amino acids. For clarity in the computation, we include one not coded amino acid, Phenylalanine, in C as well.

$$C = \begin{array}{c} \\ Cys \\ His \\ Lys \\ Phe \end{array} \begin{array}{c} \begin{array}{cccccccc} UGU & UGC & CAU & CAC & AAA & AAG & UUU & UUC \end{array} \\ \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \end{bmatrix} \end{array} \tag{2}$$

Multiplying the input matrix by the transpose of the encoding matrix yields the amino-acid identity for each input codon. We denote this intermediate matrix as E:

$$E = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \cdot \begin{array}{c} \begin{array}{cccc} Cys & His & Lys & Phe \end{array} \\ \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \end{array} = \begin{array}{c} \\ input1 : AAG \\ input2 : UGC \\ input3 : CAU \end{array} \begin{array}{c} \begin{array}{cccc} Cys & His & Lys & Phe \end{array} \\ \begin{bmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \end{array} \tag{3}$$

We then multiply E by C to obtain a mask over codons that are synonymous with the amino acid encoded by each input position. The resulting matrix is denoted as P.

$$P = \begin{array}{c} \begin{array}{cccc} Cys & His & Lys & Phe \end{array} \\ \begin{bmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \end{array} \cdot \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \end{bmatrix} \tag{4}$$

After the RNop forward pass, the model outputs a probability distribution over codons at each position. For illustration, we show the sub-matrix corresponding to the codons above and denote it as O.

$$O = \begin{array}{c} \\ Output1 \\ Output2 \\ Output3 \end{array} \begin{array}{c} \begin{array}{cccccccc} UGU & UGC & CAU & CAC & AAA & AAG & UUU & UUC \end{array} \\ \begin{bmatrix} 0 & 0.1 & 0.1 & 0.1 & 0.3 & 0.4 & 0 & 0 \\ 0 & 0.1 & 0.1 & 0.1 & 0.3 & 0.4 & 0 & 0 \\ 0 & 0.1 & 0.1 & 0.1 & 0.3 & 0.4 & 0 & 0 \end{bmatrix} \end{array} \tag{5}$$

We take the element-wise product between O and the synonymous-codon mask P to obtain the matrix OP:

$$\begin{aligned} OP &= \begin{array}{cccccccc} UGU & UGC & CAU & CAC & AAA & AAG & UUU & UUC \end{array} \\ &\begin{bmatrix} O_{11} * P_{11} & O_{12} * P_{12} & O_{13} * P_{13} & O_{14} * P_{14} & O_{15} * P_{15} & O_{16} * P_{16} & O_{17} * P_{17} & O_{18} * P_{18} \\ O_{21} * P_{21} & O_{22} * P_{22} & O_{23} * P_{23} & O_{24} * P_{24} & O_{25} * P_{25} & O_{26} * P_{26} & O_{27} * P_{27} & O_{28} * P_{28} \\ O_{31} * P_{31} & O_{32} * P_{32} & O_{33} * P_{33} & O_{34} * P_{34} & O_{35} * P_{35} & O_{36} * P_{36} & O_{37} * P_{37} & O_{38} * P_{38} \end{bmatrix} \\ &= \begin{bmatrix} 0 & 0 & 0 & 0 & 0.3 & 0.4 & 0 & 0 \\ 0 & 0.1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0.1 & 0.1 & 0 & 0 & 0 & 0 \end{bmatrix} \end{aligned} \tag{6}$$

Summing OP over the codon dimension yields a vector L, which measures how much probability mass the model assigns to synonymous codons for each input position.

$$L = \begin{bmatrix} \sum_j^N (O_{1j} * P_{1j}) \\ \sum_j^N (O_{2j} * P_{2j}) \\ \sum_j^N (O_{3j} * P_{3j}) \end{bmatrix} = \begin{bmatrix} 0.7 \\ 0.1 \\ 0.2 \end{bmatrix} \tag{7}$$

Because the output probabilities are normalized, GPLoss can be computed from L as:

$$GPLoss = \sum_i^N (1 - L_i) \tag{8}$$

Ambiguous nucleotides (N)

Some sequences contain ambiguous nucleotides denoted as N, indicating that a base is unknown (typically one of $\{T, C, A, G\}$ under our convention). When N appears within a codon, the corresponding codon token becomes ambiguous and can represent multiple concrete codons.

Introducing N expands the codon vocabulary from 4^3 to 5^3, which increases ambiguity in the loss computation. Below we describe two practical strategies for handling codons that contain N.

To make the discussion concrete, we add one additional input containing N and consider the extreme case NNN. The input matrix I becomes:

$$I = \begin{array}{c} \\ input1 : AAG \\ input2 : UGC \\ input3 : CAU \\ input4 : NNN \end{array} \begin{array}{c} \begin{array}{ccccccccc} UGU & UGC & CAU & CAC & AAA & AAG & UUU & UUC & NNN \end{array} \\ \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \end{array} \tag{9}$$

Since NNN may correspond to any codon (and thus any amino acid), we expand the encoding matrix C accordingly:

$$C = \begin{array}{c} \\ Cys \\ His \\ Lys \\ Phe \end{array} \begin{array}{c} \begin{array}{ccccccccc} UGU & UGC & CAU & CAC & AAA & AAG & UUU & UUC & NNN \end{array} \\ \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \end{bmatrix} \end{array} \tag{10}$$

After multiplication, the matrix E contains the implied amino-acid identities:

$$E = \begin{array}{c} \\ input1 : AAG \\ input2 : UGC \\ input3 : CAU \\ input4 : NNN \end{array} \begin{array}{c} \begin{array}{cccc} Cys & His & Lys & Phe \end{array} \\ \begin{bmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix} \end{array} \tag{11}$$

Solution A (allow N in predictions). In this option, we allow N-containing codons to appear in the model output and do not explicitly penalize them. The same workflow as the non-N case is applied, and E is multiplied by C to obtain P_A:

$$\begin{array}{c} \begin{array}{cccc} Cys & His & Lys & Phe \end{array} \\ \begin{bmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix} \end{array} \cdot \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 4 \end{bmatrix} \tag{12}$$

Note that the value 4 in the last row is incompatible with probability-like masking and would distort the loss. Therefore, before applying P_A to the model output, we clamp the values to $[0, 1]$ (via $torch.clamp()$). Assume O' is the corresponding model output:

$$O' = \begin{array}{c} \\ Output1 \\ Output2 \\ Output3 \\ Output4 \end{array} \begin{array}{c} \begin{array}{ccccccccc} UGU & UGC & CAU & CAC & AAA & AAG & UUU & UUC & NNN \end{array} \\ \begin{bmatrix} 0 & 0.1 & 0.1 & 0.1 & 0.3 & 0.2 & 0 & 0 & 0.2 \\ 0 & 0.1 & 0.1 & 0.1 & 0.3 & 0.2 & 0 & 0 & 0.2 \\ 0 & 0.1 & 0.1 & 0.1 & 0.3 & 0.2 & 0 & 0 & 0.2 \\ 0 & 0.1 & 0.1 & 0.1 & 0.3 & 0.2 & 0 & 0 & 0.2 \end{bmatrix} \end{array} \tag{13}$$

Then OP, L, and $GPLoss$ are computed as follows:

$$OP = \begin{bmatrix} 0 & 0 & 0 & 0 & 0.3 & 0.2 & 0 & 0 & 0.2 \\ 0 & 0.1 & 0 & 0 & 0 & 0 & 0 & 0 & 0.2 \\ 0 & 0 & 0.1 & 0.1 & 0 & 0 & 0 & 0 & 0.2 \\ 0 & 0.1 & 0.1 & 0.1 & 0.3 & 0.2 & 0 & 0 & 0.2 \end{bmatrix}, L = \begin{bmatrix} 0.7 \\ 0.3 \\ 0.4 \\ 1.0 \end{bmatrix}, GPLoss = \sum_{i}^{N} (1 - L_i) \tag{14}$$

This formulation reveals a drawback: if N is allowed in the prediction space, the NNN token can weaken or remove useful gradients, and it may also encourage the model to output N-containing codons even when the input codon is unambiguous. When this behavior is undesirable, we can either rely on additional loss terms or adopt Solution B below to discourage ambiguous predictions.

Solution B (discourage N in predictions). Here we still accept N in the *input*, but we discourage N in the *output*. To do so, we modify the second-step encoding matrix by removing the amino-acid associations of N-containing codons, yielding a new matrix C_B:

$$P_B = \begin{matrix} Cys \\ \\ \\ \\ \end{matrix} \begin{matrix} His \\ \\ \\ \\ \end{matrix} \begin{matrix} Lys \\ \\ \\ \\ \end{matrix} \begin{matrix} Phe \\ \\ \\ \\ \end{matrix} \begin{bmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \end{bmatrix} \tag{15}$$

With this modification, NNN no longer trivially matches every amino acid, and the model receives a meaningful gradient signal when it assigns probability mass to NNN. Using the same output O', we obtain:

$$OP = \begin{bmatrix} 0 & 0 & 0 & 0 & 0.3 & 0.2 & 0 & 0 & 0 \\ 0 & 0.1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0.1 & 0.1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0.1 & 0.1 & 0.1 & 0.3 & 0.2 & 0 & 0 & 0 \end{bmatrix}, L = \begin{bmatrix} 0.5 \\ 0.1 \\ 0.2 \\ 0.8 \end{bmatrix}, GPLoss = \sum_{i}^{N} (1 - L_i) \tag{16}$$

Solution B therefore encourages the model to avoid predicting codons containing N, leading to less ambiguous outputs. Nevertheless, we note that codons with N in the original input may be problematic for downstream biological experiments and should be treated with caution.

Formulation of GPLoss

Because each position corresponds to a normalized codon distribution, the per-position GPLoss score is bounded by 1. We summarize the computation in Eq. (17). Here, **O,P** denotes the model output and the synonymous-codon mask (as above), and **P** denotes the encoding matrix. $L_{GP,i}$ is the per-position synonymous-mass score, and $GPLoss$ is the total loss. In our experiments, the GPLoss weight is set to 10^{-1} to achieve a zero mutation rate.

$$\mathbf{OP} = sum(\mathbf{O} \cdot (\mathbf{P})) = \begin{bmatrix} L_{GP,1} \\ L_{GP,2} \\ ... \\ L_{GP,n} \end{bmatrix}, GPLoss = \sum_{i=1}^{n} (1 - L_{GP,i}) \tag{17}$$

1.1.7. CAILoss: Improving CAI by Penalizing Low CAI Codons

We leverage the CAI [39] to design CAILoss, which explicitly injects codon-usage-bias knowledge into RNop. Intuitively, CAILoss evaluates the CAI scores of the model's output distribution at each codon position and penalizes probability mass assigned to low-CAI codons, thereby encouraging the selection of codons preferred by the target species.

The CAILoss parameters are initialized from species-specific codon-usage tables. We downloaded the raw data from the open-source GenScript database [12]. Currently, we have incorporated parameters for more than 10 species, and additional species can be added by including their codon-usage-bias profiles.

Following the same workflow as GPLoss, we replace the binary entries in the encoding matrix with codon-specific CAI values, yielding a CAI-weighted matrix $C_{CAI-raw}$:

$$C_{CAI-raw} = \begin{array}{c} \\ Cys \\ His \\ Lys \\ Phe \end{array} \begin{array}{c} \begin{array}{ccccccccc} UGU & UGC & CAU & CAC & AAA & AAG & UUU & UUC & NNN \end{array} \\ \begin{bmatrix} 0.46 & 0.54 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0.57 & 0.43 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0.74 & 0.26 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0.58 & 0.42 & 0 \end{bmatrix} \end{array} \tag{18}$$

Using raw CAI values directly would not ensure that the optimal choice yields a per-position score of 1. Therefore, we normalize the CAI values within each amino-acid group by dividing by the maximum CAI among synonymous codons to form a matrix C_{CAI}:

$$C_{CAI} = \begin{array}{c} \\ Cys \\ His \\ Lys \\ Phe \end{array} \begin{array}{c} \begin{array}{ccccccccc} UGU & UGC & CAU & CAC & AAA & AAG & UUU & UUC & NNN \end{array} \\ \begin{bmatrix} \frac{0.46}{0.54} & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & \frac{0.43}{0.57} & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & \frac{0.26}{0.74} & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & \frac{0.42}{0.58} & 0 \end{bmatrix} \end{array} \tag{19}$$

We then apply C_{CAI} within the same two-step workflow as GPLoss: the input is first processed by the original matrix C to obtain the amino-acid identity, and C_{CAI} is applied in the second multiplication step to weight synonymous codons by their normalized CAI values. Finally, CAILoss follows the same aggregation as GPLoss to produce the scalar loss. The computation is summarized in Eq. (20).

$$L = sum(\mathbf{O} \cdot (\mathbf{I} \cdot \mathbf{C}^T \cdot \mathbf{C}_{CAI})) = \begin{bmatrix} L_{CAI,1} \\ L_{CAI,2} \\ ... \\ L_{CAI,n} \end{bmatrix}, CAILoss = \sum_{i=1}^{n} (1 - L_{CAI,i}) \tag{20}$$

1.1.8. tAILoss: Improving tAI by Penalizing Low tAI Codons

The tAILoss targets translation efficiency by incorporating tRNA availability information across species. Following the tAI formulation in prior work [36, 37], codons whose corresponding anticodons have higher tRNA copy numbers are treated as more preferred. We adapt the original tAI computation to match our coding rule by re-ordering codons to align with our input/output matrix convention and mapping anticodon statistics to their corresponding codons.

In practice, tAILoss penalizes probability mass assigned to codons associated with low tRNA availability; increasing tAI is expected to improve translation elongation efficiency. We downloaded the raw tRNA data from GtRNAdb [4]. Currently, we have incorporated tAILoss parameters for 6 common species, and additional species can be added by providing their tRNA copy-number profiles.

The tAILoss parameters form a vector: $tAI\vec{param} - raw$. The original tAI computation combines tRNA counts (and their logarithms) with codon counts, which makes the scale dependent on sequence length and complicates normalization.

$$tAI\vec{param} - raw = \begin{array}{c} \begin{array}{ccccccccc} UGU & UGC & CAU & CAC & AAA & AAG & UUU & UUC & NNN \end{array} \\ \begin{bmatrix} 2.36 & 4 & 4.72 & 8 & 8 & 16.56 & 11. & 6.49 & 0 \end{bmatrix} \end{array} \tag{21}$$

Therefore, analogous to CAILoss, we normalize the parameters within each synonymous codon set by dividing by the maximum value to form a vector $tAI\vec{param}$:

$$tAI\vec{param} = \begin{array}{c} \begin{array}{ccccccccc} UGU & UGC & CAU & CAC & AAA & AAG & UUU & UUC & NNN \end{array} \\ \begin{bmatrix} 0.59 & 1 & 0.59 & 1 & 0.48 & 1 & 0.59 & 1 & 0 \end{bmatrix} \end{array} \tag{22}$$

We further set the start codon and stop codon parameters to 1.0 to prevent the model from altering these special tokens. The calculation of tAILoss for input above can be written as:

$$
\begin{aligned}
L &= exp(\mathbf{O} \cdot log(tAI\vec{param})^T) \\
&= \begin{bmatrix} 0 & 0.1 & 0.1 & 0.1 & 0.3 & 0.2 & 0 & 0 & 0.2 \\ 0 & 0.2 & 0.1 & 0.1 & 0.3 & 0.1 & 0 & 0 & 0.2 \\ 0 & 0.1 & 0.2 & 0.1 & 0.3 & 0.2 & 0 & 0 & 0.1 \\ 0 & 0.1 & 0.1 & 0.3 & 0.1 & 0.2 & 0 & 0 & 0.2 \end{bmatrix} \cdot \begin{bmatrix} 0.59 \\ 1 \\ 0.59 \\ 1 \\ 0.48 \\ 1 \\ 0.59 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0.7610 \\ 0.7610 \\ 0.7223 \\ 0.8815 \end{bmatrix}, \\
tAILoss &= \sum_i^N (1 - L_i)
\end{aligned} \tag{23}
$$

Eq. (24) summarizes the tAILoss computation, where $tAI\vec{param}$ denotes the tAI parameter vector.

$$
L = exp(\mathbf{O} \cdot log(tAI\vec{param}^T)) = \begin{bmatrix} L_{tAI,1} \\ L_{tAI,2} \\ ... \\ L_{tAI,n} \end{bmatrix}, tAILoss = \sum_i^N (1 - L_{tAI,i}) \tag{24}
$$

1.1.9. MFELoss: Full Sequence Optimization for MFE

To improve the minimum free energy of mRNA sequences, we designed the MFELoss to incorporate the knowledge of mRNA stability and half-life into the RNop model. Generally, the lower the MFE of mRNA sequences is, the more stable the mRNA sequences are, and the longer their half-life is, which indicates better expression time and amount in the cell[27, 25].

To calculate the MFE of mRNA sequences, there are methods using algorithms which are the $O(N^3)$ time complexity [26, 45]. These methods can hardly be integrated into neural networks due to time issues–their time consumption is too high for deep learning iterations. Therefore, to make MFE computation fast and utilizable in deep learning, we build a small loss network specialized to predict the MFE of mRNA sequences.

First, we calculate the MFE value in kJ/mol using RNAStructure[38] for all RNA sequences in our collected datasets. A very small Multilayer Perceptron (MLP) neural network is applied for rapid estimation of mRNA sequences. We tried different model sizes and found that a small model with 12 layers and 5.4M parameters can achieve high precision, up to $R^2 = 0.9983$ on the test set. The calculation of MFELoss is shown in the Eq.25. The $MFE_{i,gt}$ represents the ground truth MFE of the sequence i, and OP_i represents the ith output of the optimization model. The F_{mfe} means the OP_i is processed by the MFE model mentioned above. The $\lambda \in [1, +\infty)$ is the MFE ratio to ensure the optimization target is higher than the original MFE. Consequently, the loss operation $Loss$, which can be selected from L1, SmoothL1, MSE, and Charbonnier loss, is applied to these vectors.

$$
MFELoss = Loss(\lambda \times MFE_{i,gt}, F_{mfe}(Seq_i)) \tag{25}
$$

The MFELoss makes the optimization model prone to generating mRNA sequences with better (lower) MFE. Meanwhile, the MFE deviation values of the MFELoss are usually in the 10^2 order of magnitude, while others are within $(0, 1]$. Therefore, the default loss weight of the MFELoss is set to 10^{-2}.

1.1.10. Implementation Details

During training, the RNop model is applied with PyTorch version 2.1 and trained with CUDA version 12.2. We train each model for 100,000 iterations (equivalently 10 epochs) with a batch size of 64 and a learning rate of 10^{-4} with cosine decay. The AdamW [16, 23] optimizer is applied with a 1000 iterations warm-up. All experiments are trained using a single NVIDIA GeForce A800 GPU, running for 18 hours to complete the training process. The models are trained on the training dataset and validated on the validation dataset during the training process, and the test dataset is independent in the subsequent test process.

1.1.11. Quantitative Metrics

We evaluate optimized sequences using three biological metrics (CAI, tAI, MFE) and a set of error-based metrics that quantify fidelity relative to the original sequences.

The CAI is a widely used measure of how well codon usage in an mRNA matches the codon-usage bias of a target species. We compute CAI for both original and optimized sequences to assess their codon optimality in the target species, following [39, 12].

$$CAI = f_{CAI}(mRNASequence). \tag{26}$$

The tAI evaluates how well an mRNA sequence matches the tRNA pool of a species via tRNA anticodon counts. We compute tAI for the output sequences to quantify their tRNA affinity in the target species, following [36, 37].

$$tAI = f_{tAI}(mRNASequence). \tag{27}$$

The MFE is a thermodynamic descriptor of mRNA structure; lower MFE has been associated with increased stability and longer half-lives [27, 25]. We use RNAStructure [38] to compute the MFE of optimized mRNA sequences.

$$MFE = f_{MFE}(mRNASequence). \tag{28}$$

To characterize discrepancies between original and optimized RNA sequences, we further define four error metrics: length error rate (LER), codon error rate (CER), codon error percentage (CEP), and sequence difference percentage (Sequence Difference).

The **length error rate (LER)** is the fraction of sequences whose optimized length differs from the original (indicating missing or extra codons, which are common in probabilistic models). Let N be the total number of sequences and N_{LE} the number with length errors; then LER is given in Eq. 29.

$$LER = \frac{N_{LE}}{N}. \tag{29}$$

The **codon error rate (CER)** is the proportion of sequences that contain at least one incorrect codon, and the **codon error percentage (CEP)** is the average fraction of incorrect codons among those erroneous sequences. Let N_{CE} (out of N total sequences) denote the number of sequences with codons that do not preserve the original amino-acid sequence, and let ce_i of n_i codons be wrong in the i-th such sequence for $i \in [1, N_{CE}]$. CER and CEP are defined in Eqs. 30 and 31.

$$CER = \frac{N_{CE}}{N}. \tag{30}$$

$$CEP = \frac{\sum_i^{N_{CE}} ce_i}{\sum_i^{N_{CE}} n_i}. \tag{31}$$

The **sequence difference percentage (Sequence Difference)** measures how strongly error-free sequences are altered at the codon level. Let N_{NE} be the number of sequences without length or codon errors, so that $N = N_{LE} + N_{CE} + N_{NE}$. For each such sequence $j \in [1, N_{NE}]$, let sd_j of n_j codons differ from the original while remaining correct (synonymous). Sequence Difference is defined in Eq. 32.

$$SequenceDifference = \frac{\sum_j^{N_{NE}} sd_j}{\sum_j^{N_{NE}} n_j}. \tag{32}$$

When LER is nonzero, CER, CEP, and Sequence Difference are computed only over sequences without length errors, since differing lengths make a codon-wise comparison ill-defined.

2. Results

2.1. RNop Simultaneously Achieves Multi-Objective Optimization and Sequence Fidelity *in Silico*

In the introduction, the trade-off between **sequence fidelity**, **multi-objective optimization**, and **computational efficiency** was defined as the "impossible triangle". We first sought to validate the ability of RNop to conquer the first two of these challenges through large-scale *in silico* experiments.

Our first experiment was designed to evaluate RNop's ability to achieve multi-objective optimization in biological metrics while strictly maintaining sequence fidelity. We trained RNop models on the training dataset with over 5,400,000 sequences, targeting different species: "E. coli", "Mouse", "Chinese hamster", and "Human". These models were then benchmarked on a test set of 540,000 sequences. The dataset is preprocessed to avoid data leakage, so all sequences are "unseen" for pretrained models. The calculation process adheres to the methods outlined in the original papers[39, 36, 37].

The validation results confirm that RNop successfully achieves perfect sequence fidelity, producing **no undesired amino acid mutations** across the test set and validation set. Meanwhile, the model demonstrated consistent **multi-objective optimization**. As visualized in the violin plots in Fig. 3, sequences optimized by RNop exhibited significantly higher overall CAI and tAI values and significantly lower MFE values compared to the original across all four parallel experiments. This proved RNop's capacity to simultaneously optimize multiple biological objectives targeting various species.

Next, we substantiated that our knowledge-infusion framework was genuinely effective in the second *in silico* experiment. A model that truly learned the implicit rules of optimization should not only optimize sequences from heterologous species but also further improve homologous sequences. Therefore, we used the species-specified test set, and the results are shown in Fig. 4. When the target species was set to "E. coli", RNop improved metrics for sequences from all species, including virus, mouse, human, and E.coli homologous sequences. This is also consistent for models targeting "Mouse" (Fig. 4 (b)) and "Human" (Fig. 4 (c)).

These results substantiate the generalization ability of the RNop model. It shows the advantage of the knowledge-infusion schema over other methods that learn directly from natural sequences. RNop is not memorizing or imitating natural sequences from a specific species; rather, it has learned the explicit biological prior knowledge for that species. Therefore, RNop can accept a sequence from any resource and exhibit robust performance in both heterologous and homologous optimization.

2.2. RNop provides high-throughput mRNA optimization

The computation efficiency is another crucial piece of the "impossible triangle" of mRNA optimization. Current optimization methods, such as dynamic programming (DP), have high time complexity, $O(n^3)$ [41, 6], leading to burdensome runtimes for sequences longer than 2,000 nucleotides s (nt). Even faster approaches like LinearDesign [45] have $O(n^2)$ complexity, and their processing time typically becomes prominent for sequences exceeding 3,000 nt.

By contrast, RNop own innate advantages for its deep learning architecture in rapid inference and batch processing. The processing time of RNop is tied to the parameter amount and model depth configured at training and does not change afterward. For an input length within the model's capacity, sequences will be processed in essentially the same time. Meanwhile, the batch processing capability enables RNop to process multiple sequences in parallel. Consequently, RNop can offer a strong advantage in most scenarios due to low per-sequence latency and efficient batch processing.

We evaluate inference time across RNop models with different embedding dimensions, depths, and sequence lengths. The model is running on one Nvidia A800 GPU, with CUDA version 12.2. The results are shown in Fig.7 (b). For comparison, CDSFold [41] requires about 1 hour to design a 2700 nt CDS, and LinearDesign takes minutes to optimize a 2048 nt mRNA, and RNop models achieve throughput up to 44.49 sequences per second for any sequence under 3072 nt. Though larger models handling longer sequences need more time, they also achieve better performance than the aforementioned methods. The higher processing speed enables high-throughput mRNA design and facilitates integration with protein structure and function prediction models.

2.3. Knowledge-Infused Loss Functions Provides Interpretable Control

Having addressed the "impossible triangle" and validated RNop in biological experiments, we sought deeper insights into the functions of each type of loss function about how they contribute to the optimization process. This exploration is demonstrated by the models with different weights of loss functions, through which we can observe the effect of each loss function on the target sequences.

Controllable Sequence Fidelity of GPLoss

The GPLoss is designed to avoid undesired amino acid (aa) mutation. The mutation here means the protein aa sequences before and after optimization are not synonymous. Without constraint, most deep learning and neural network models may cause mutations, as they are probabilistic models. Some deep learning methods [11] replace the mutated codons with the original ones to avoid mutation, but this may cause the loss of optimality. Therefore, we designed the GPLoss to avoid mutation generation without post-processing.

We conducted experiments on the RNop model with different GPLoss weights ranging from 1 to 10^{-6} to showcase its controllable flexibility. We can find that the mutation rate is actually adjustable in the RNop model, and it varies with the GPLoss weight. The results, summarized in Fig.7 (a), show that high loss weights preserve the synonymous protein sequences after optimization. For weights $\geq 10^{-2}$, the mutation rate is tightly controlled, and neither codon errors nor length errors occur. At the same time, GPLoss offers controllable flexibility: by tuning the loss weight, the mutation rate of optimized sequences can be adjusted from 0% to nearly 100%. As the loss weight decreases, the mutation rate increases; at 10^{-6}, widespread mutations are observed in the output sequences.

These results can explain the role of GPLoss in regulating mutation behavior in the RNop models.

Modulation of Codon Usage by CAILoss and tAILoss

Biologically, the codon usage is not arbitrary in different species. To manifest this preference, the CAI[39] is proposed to measure the codon usage bias. Meanwhile, the tAILoss specifically focuses on tRNA counts across different species, where the anticodons associated with the highest tRNA counts are considered the most preferred codons. The CAILoss and tAILoss are supposed to regulate the CAI and tAI of output sequences.

We conducted experiments adjusting the loss weight of CAILoss and tAILoss from 10^{-1} to 10^{-5}. In these experiments, CAILoss and tAILoss were independently evaluated. The results are presented in Fig.7 (c) and (d). Mostly, the change in the loss weight of CAILoss leads to a slight corresponding change in the CAI values of output sequences. However, though tAI manifests a reduction trend when the loss weight of tAILoss is reduced, when the loss weight is set to 10^{-2} or 10^{-1}, the increase in the loss weight for tAILoss leads to overfitting in some experiment runs.

This trend can explicitly prove that adjusting the loss weight of CAILoss and tAILoss during training can regulate the CAI and tAI of output sequences. Meanwhile, we also note that the loss weights shall balance the contribution of CAILoss and tAILoss. Exaggerating the loss weight of one loss function may cause overfitting or unstable convergence.

Explicit Optimization of MFE by MFELoss

MFE profoundly affects the stability of mRNA and the efficiency of translation. Though improvements in CAI and tAI can also reduce MFE [33], we conducted experiments to isolate the contribution of MFELoss. We compare the MFE of sequences optimized by RNop with and without MFELoss against the original sequences.

As shown in Figure 6, RNop models with MFELoss produce sequences with lower MFE than both the original sequences and those optimized without MFELoss. This pattern holds across target species (“E. coli,” “Mouse,” “Chinese hamster,” and “Human”). Notably, sequences optimized without MFELoss also exhibit lower MFE than the originals, consistent with prior findings [33]. These results demonstrate that MFELoss effectively improves the MFE of mRNA sequences beyond gains attributable to CAI/tAI alone.

We also introduce a parameter λ to ensure the optimization target is higher than the original MFE by setting a target MFE lower than the original. The higher ratio λ can set the lower target MFE. We conducted experiments on models with λ values from 0.9 to 1.5, and the results are summarized in Fig.7 (e).

We observe that $\lambda = 1.1$ provides the best balance, yielding the largest MFE improvement alongside strong performance on other metrics. When λ is too high, low MFE is overemphasized: the target may exceed the maximum attainable improvement for sequences, leading to persistent MFELoss pressure that competes with other objectives and, thus, causes length or codon errors (as seen for $\lambda = 1.5$). Conversely, when λ is too low, the target is easily satisfied, and the MFE will not be optimized. As a result, sequences exhibit relatively higher MFE. Therefore, we can conclude that the MFELoss ratio λ is an important hyperparameter that shall be set within the maximum optimizable capability for sequences to balance the MFE improvement and model convergence.

2.4. Sequences optimized by RNop model exhibit higher expression efficiency in *in vitro* experiments

Our *in silico* experiments demonstrated that RNop successfully addresses the "impossible triangle". However, the golden standard of mRNA optimization is the protein expression rate within a complex biological environment. We conducted biological experiments to demonstrate the *in vitro* effectiveness of RNop and confirm its ability to preserve protein function.

We selected two of the most widely used and highly optimized reporter proteins: the enhanced green fluorescent protein (EGFP) and the enhanced cyan fluorescent protein (ECFP). As these sequences are already well-studied and optimized, any significant improvement is exceptionally difficult to achieve. Meanwhile, a sequence of a key antigen protein, representing a potential component for mRNA vaccine production, was selected as the target protein to bridge the gap between model proteins and real-world applications in the final experiments.

The optimized sequences were compared against control groups: The tested mRNA sequences were labeled as: NC (negative control), HO (original Human-Optimized), LD (LinearDesign) [45], and three variants from our RNop model grid with different loss weights: RNop (Standard RNop model); RNop-C (RNop model optimized with a 10-fold increase in CAILoss weight); and RNop-T (RNop model optimized with a 10-fold increase in tAILoss weight). These sequences are transfected and then expressed *in vitro* with the methods explained in the Section.Method. The expression rate was measured through the fluorescent enzyme-labeled instrument and western blot at 24 hours and 48 hours. The number of total replications is 3, and the significance levels are $* \ p < 0.05$, $** \ p < 0.01$, $*** \ p < 0.001$.

Results of EGFP optimization demonstrated the effectiveness of RNop:

In the first experiment, we use the most widely used fluorescence protein, the EGFP. The results are presented in Fig.5 (a-e).

The fluorescence measurement results (Fig.5 (b,d)) show a clear advantage of RNop over other groups. At both 24 and 48 hours post-transfection, variants of RNop consistently achieved the highest fluorescence intensity, indicating high protein expression levels. Its performance significantly surpassed the compared groups. The RNop-C variant is the best optimization model for EGFP; its optimized sequence yielded up to 2.28-fold higher expression than the HO group at 48 hours. Through the western blot results (Fig.5 (c,e)), we can draw a similar conclusion, with RNop variants yielding higher expression than LD and HO. These results demonstrate RNop's ability to obtain substantial performance gains even from already well-optimized sequences.

Results of ECFP optimization showed the robustness:

We extended the validation to the ECFP, and the results are shown in Fig.5 (f-j).

RNop variants again outperformed other groups significantly, exhibiting substantial increases relative to the original HO sequence. Meanwhile, the LinearDesign optimized sequence suffered an expression failure: protein expression and fluorescence levels were near 0 or near the NC group at both 24 and 48 hours.

The failure in the LinearDesign group can showcase the concept of the "impossible triangle." When optimizing CAI and MFE, LinearDesign may have introduced excessive secondary structure modifications. Visualization analysis suggests the ECFP sequence was modified into a structure with nearly all nucleotides paired. This structure may hinder translation initiation or elongation and ultimately cause expression failure. Conversely, the multiple optimization objectives of the RNop framework prevent the domination and over-emphasis of any single factor, and thus avoid the expression failure. This highlighted the critical role of sequence fidelity and balanced multi-objective optimization, which are the key strengths of the RNop framework.

Optimization of target protein demonstrated the versatility :

Then, we applied RNop to optimize the sequence of a key antigen protein, a component of an mRNA vaccine. This experiment can demonstrate the effectiveness of RNop on a sequence directly relevant to therapeutic and industrial goals.

The results of the target protein are shown in Figure 5 (k,l). In these experiments, all optimization methods yielded significant expression increases compared to the original. The standard RNop once again achieved the highest protein expression levels at 40 hours post-transfection, outperforming LD. Computational analysis indicated that the optimized sequence of RNop and LD has moderate structural changes. The similar structure with HO permitted the protein expression, yet the predicted MFE of the LD sequence suggested insufficient optimization in MFE compared to the sequences optimized by our method.

In conclusion, these *in vitro* experiments, including highly optimized reporters and a therapeutically relevant antigen, validate the effectiveness of RNop. RNop models consistently delivered optimized sequences yielding superior protein expression compared to both original sequences and the state-of-the-art LinearDesign algorithm, while maintaining crucial sequence fidelity and avoiding expression failures caused by unbalanced optimization.

2.5. Summarization: The Controllable Solution of the "Impossible Triangle"

In summary, the *in silico*, *in vitro*, and loss function analyses demonstrate that RNop successfully resolves the "impossible triangle" situation of current mRNA optimization methods.

The *in silico* experiments confirmed RNop's ability to achieve robust multi-objective optimization while strictly maintaining perfect sequence fidelity. Meanwhile, *in vitro* validation confirmed these computational improvements can translate to real-world protein expression (up to 2.28-fold). Our RNop models also avoided the expression failures associated with unbalanced optimization and outperformed SOTA methods. Furthermore, RNop accomplishes all of these while achieving high computational efficiency, enabling high-throughput applications.

Furthermore, the loss function analyses provide the explicit explanation for the controllability of RNop: the K-iL framework is not a "black box". Each loss function provides interpretable and tunable control over its targeted biological objective. After having validated RNop's performance and mechanism, we will now discuss the broader implications of these findings.

3. Discussion

In the *in vitro* experiments, the expression failure of LinearDesign optimized ECFP is an example explaining why multi-factor optimization is important. The overemphasized optimization factor would lead to fatal failure. However, the RNop framework avoided this problem with its balanced and biological knowledge-infused approach. Meanwhile, the sequence fidelity has been a problem for probabilistic models, while the RNop framework achieves zero mutation rate and codon error, reaching the same level as algorithm methods. These advantages, working with the innate advantages of deep learning: batch processing and rapid inference, make RNop outstanding among current deep-learning-based methods.

Controllable synonymous retention by K-iL framework. Many deep-learning-based codon optimization pipelines handle protein mutations via post-processing: after decoding, non-synonymous codons are replaced by the original codons to restore the amino-acid sequence [11]. While this can remove mutations, it breaks the optimized distribution and may undo improvements in other optimization objectives. In contrast, RNop enforces synonymy during optimization: GPLoss imposes a differentiable, matrix-based constraint directly on the codon probability space. As a result, strict synonymous outputs can be achieved without post-hoc repair, and the strength of this constraint can be continuously tuned via the GPLoss weight—yielding either strict synonymy with high loss weight or controlled flexibility with low loss weight when desired.

Knowledge-infused learning: from physics to biology. Physics-informed neural networks (PINNs) [35] have demonstrated how encoding physical laws into loss terms can guide neural networks to solve partial differential equations and related inverse problems. RNop extends this knowledge-infused paradigm to biological sequence design: instead of PDE residuals or conservation laws, we encode biological priors such as codon–amino-acid mappings, codon-usage bias, tRNA adaptation, and structural stability as mechanism-aligned loss functions for mRNA optimization: CAILoss, tAILoss, and MFELoss. This cross-domain perspective suggests that K-iL is a general strategy for structuring learning problems in complex scientific domains, not limited to physics but also effective for high-dimensional biological sequence design.

Explainability and controllable optimization. The explainability is another major problem for mRNA optimization. The former attempts in deep learning based models, such as works of Gong et al.[13, 44], Goulet et al.[14], and Jain et al.[15], followed the strategy of learning from sequences of specific species and assumed the natural mRNA sequences contain the best pattern of possible sequences. However, they hardly discussed the rational reasons for this approach. Gong et al.[44] also claimed that the CDS optimization method in the GEMORNA approach that they had proposed is likely to be a black box, and the explainability of this model is limited for the implicitly learned features.

On the contrary, the knowledge-infused RNop framework provides a more interpretable explanation for the optimization process: all learned features are explicit and can be explained through biological findings. [39, 36, 37, 24, 26] The K-iL framework directly addresses the "black box" problem. As our loss function studies demonstrated, the ability to modulate the contribution of each loss function offers a clear view into the optimization process and enables controllable mRNA design. Thus, the RNop framework provides a powerful approach to make deep learning-based mRNA optimization get rid of the limitations of implicit and "black-box" learning from natural sequences.

Effects of different backbone architectures on RNop. We chose ViT as our cornerstone model, which seems unusual. However, the choice of the ViT model is not an arbitrary decision but a critical, rationally-driven decision to enable our K-iL framework according to experiments. In our exploration of cornerstone architectures for mRNA sequence optimization, we systematically evaluated linear networks[17, 42], sequence-to-sequence models[11, 14], Transformer-based language models[9, 19, 34, 7], and Vision Transformers (ViT)[10, 22]. As detailed in Supplementary information, alternative architectures all failed: linear models and Transformers (BERT, T5) consistently

suffered from overfitting, while LSTMs were structurally incompatible with full-sequence losses like MFELoss. The ViT architecture uniquely provided the necessary parameter-efficiency and global dependency modeling required to balance all four knowledge-infused losses simultaneously. The ViT functions as an extraordinary foundation for RNop, inspiring us that the success of the K-iL framework depends not just on the prior knowledge, but on a corresponding adequate architecture to support it.

Modular framework of RNop enables future incorporation of UTR generation. The UTR design is deliberately decoupled from our study on CDS optimization, which is a methodological choice. Unlike the biological priors we have mentioned governing the CDS, UTR efficiency is governed by fundamentally distinct factors like RBP binding sites, IRES elements, and miRNA avoidance. [13, 21, 20, 5] Logically speaking, we can regard UTR design as a totally different task from CDS optimization. Forcing these two disparate tasks of different biological logic into a single end-to-end model will risk creating a new, more complex "black box." It may also make it impossible to disentangle which factors contribute to the final expression.

Meanwhile, UTR generation operates at the nucleotide level, using a different vocabulary from the codon-based RNop model. Our coding rules for CDS optimization are designed for codon-level tokenization. Incorporating UTR generation would require expanding the vocabulary to single nucleotides, potentially introducing instability in the current framework. Also, to preserve simplicity and efficiency, we prefer to separate UTR generation from the RNop implementation.

Our K-iL approach, by its very nature, demands modularity and interpretability. By separating these components, we ensure that the optimization of the CDS is mechanistically transparent. This modularity is an advantage for integrating future works. It creates a potential pathway to develop a separate compatible K-iL module for UTRs. The methodology applied in RNop allows us to rationally combine an optimized CDS with optimized UTR components, from 5' UTR to poly(A) tail, rather than relying on a whole but "black box" optimization.

RNop is ready for real world protein production. The experiments on the target antigen protein for mRNA vaccine also provide a meaningful insight into the potential of RNop models in protein production. The mRNA vaccine component can be a perfect representation of all other proteins, not only in mRNA vaccines, but also the proteins in industry, medicine, and laboratory that require high expression in target species. RNop enables the optimization across a large scope of species, thus, the protein expression system is not limited to human cells. It can boost the production through yeast and E. coli, the expression in Chinese Hamster Ovary (CHO) cells, and other systems that can be utilized in protein production. The RNop can work as a pivot for the mRNA optimization that empowers the heterologous protein to be easily expressed from one species to another. Especially, due to the rapid response speed, the RNop can provide optimized sequences with unparalleled low-cost and high-throughput.

4. conclusion

In conclusion, we demonstrate that the "impossible triangle" of mRNA optimization—the long-held trade-off between sequence fidelity, multi-objective capability, and computational efficiency—is not a fundamental limit but a solvable engineering challenge. The RNop framework successfully solves this challenge by integrating biological prior knowledge into a Transformer architecture in an extensible and modular design. More importantly, this work represents an impactful shift in mRNA optimization methodology. By making knowledge infusion explicit, controllable, and interpretable, RNop begins to transform mRNA design from a "black-box" operation to a predictable and explainable engineering discipline.

The true potential of RNop lies in the modularity we developed in its conceptualization. The RNop framework is not a static solution but a dynamic platform. New biological insights—such as the Mean Typical Decoding Rate (MTDR) [8] or tissue-specific tRNA profiles—can be added as plug-in, mechanism-aligned loss functions, enabling targeted optimization objectives without redesigning the backbone model. In parallel, expanding to extra species is straightforward in this platform setting: species-specific priors like CAI and tAI can be incorporated by updating the corresponding loss-parameter tables. This modularity also supports future integration of a compatible UTR module, allowing more rational end-to-end engineering of complete mRNAs, from 5' UTR to poly(A) tail.

The impact of RNop can extend far beyond mRNA. The core principle of RNop— an approach infusing biological knowledge into deep learning models —provides a powerful template for other complex challenges in computational biology. This strategy can be applied to synthetic genome generation, protein design, and biological dark matter exploration. By demonstrating how to guide deep learning with the clarity of biological knowledge, RNop provides a scalable path to engineering biology rather than exploring in the dark.

Acknowledgements

The authors would like to thank Prof. Sanyang Han and Dr. Hongtian Yang from Tsinghua University, Tsinghua Shenzhen International Graduate School, and Prof. Yan Zhao from Sun Yat-sen University, School of Chemical Engineering and Technology for sharing their valuable experience.

Competing interests: The authors declare no competing interests.

Author contributions: **Zheng Gong:** Conceptualization, Methodology, Validation, Data Curation, Writing - Original Draft, Visualization. **Ziyi Jiang:** Conceptualization, Validation, Writing - Review and Editing. **Weihao Gao:** Conceptualization, Writing - Review and Editing. **Yuanyuan Wang:** Validation. **Zhining Cai:** Validation. **Zhuo Deng:** Writing - Review and Editing. **Lan Ma:** Funding acquisition, Supervision.

Data statement: The data and code are available at `https://github.com/HudenJear/RPLoss` for justified usage and research upon request. Please contact the corresponding author if you have any questions about the data and code.

A. My Appendix

Appendix sections are coded under `\appendix`.

`\printcredits` command is used after appendix sections to list author credit taxonomy contribution roles tagged using `\credit` in frontmatter.

CRediT authorship contribution statement

Zheng Gong: Conceptualization, Methodology, Validation, Data Curation, Writing - Original Draft, Visualization. **Ziyi Jiang:** Conceptualization, Validation, Writing - Review and Editing. **Weihao Gao:** Conceptualization, Writing - Review and Editing. **Yuanyuan Wang:** Validation. **Zhining Cai:** Validation. **Zhuo Deng:** Writing - Review and Editing. **Lan Ma:** Funding acquisition, Supervision.

References

[1] Babjac, A.N., Lu, Z., Emrich, S.J., 2023. Codonbert: Using bert for sentiment analysis to better predict genes with low expression, in: Proceedings of the 14th ACM International Conference on Bioinformatics, Computational Biology, and Health Informatics, Association for Computing Machinery, New York, NY, USA. URL: https://doi.org/10.1145/3584371.3613013, doi:10.1145/3584371.3613013.

[2] Brixi, G., Durrant, M.G., Ku, J., Poli, M., Brockman, G., Chang, D., Gonzalez, G.A., King, S.H., Li, D.B., Merchant, A.T., Naghipourfar, M., Nguyen, E., Ricci-Tam, C., Romero, D.W., Sun, G., Taghibakshi, A., Vorontsov, A., Yang, B., Deng, M., Gorton, L., Nguyen, N., Wang, N.K., Adams, E., Baccus, S.A., Dillmann, S., Ermon, S., Guo, D., Ilango, R., Janik, K., Lu, A.X., Mehta, R., Mofrad, M.R., Ng, M.Y., Pannu, J., Ré, C., Schmok, J.C., John, J.S., Sullivan, J., Zhu, K., Zynda, G., Balsam, D., Collison, P., Costa, A.B., Hernandez-Boussard, T., Ho, E., Liu, M.Y., McGrath, T., Powell, K., Burke, D.P., Goodarzi, H., Hsu, P.D., Hie, B.L., 2025. Genome modeling and design across all domains of life with evo 2. bioRxiv URL: https://www.biorxiv.org/content/early/2025/02/21/2025.02.18.638918, doi:10.1101/2025.02.18.638918, arXiv:https://www.biorxiv.org/content/early/2025/02/21/2025.02.18.638918.full.pdf.

[3] Buhr, F., Jha, S., Thommen, M., Mittelstaet, J., Kutz, F., Schwalbe, H., Rodnina, M.V., Komar, A.A., 2016. Synonymous codons direct cotranslational folding toward different protein conformations. Molecular Cell 61, 341–351. URL: https://www.sciencedirect.com/science/article/pii/S1097276516000095, doi:https://doi.org/10.1016/j.molcel.2016.01.008.

[4] Chan, P.P., Lowe, T.M., 2015. GtRNAdb 2.0: an expanded database of transfer RNA genes identified in complete and draft genomes. Nucleic Acids Res 44, D184–9.

[5] Chu, Y., Yu, D., Li, Y., Huang, K., Shen, Y., Cong, L., Zhang, J., Wang, M., 2024. A 5' utr language model for decoding untranslated regions of mrna and function predictions. Nature Machine Intelligence 6, 449–460. URL: https://doi.org/10.1038/s42256-024-00823-9, doi:10.1038/s42256-024-00823-9.

[6] Cohen, B., Skiena, S., 2003. Natural selection and algorithmic design of mrna. Journal of Computational Biology 10, 419–432. URL: https://doi.org/10.1089/10665270360688101, doi:10.1089/10665270360688101, arXiv:https://doi.org/10.1089/10665270360688101. pMID: 12935336.

[7] Dai, Z., Yang, Z., Yang, Y., Carbonell, J., Le, Q.V., Salakhutdinov, R., 2019. Transformer-xl: Attentive language models beyond a fixed-length context. URL: https://arxiv.org/abs/1901.02860, arXiv:1901.02860.

[8] Dana, A., Tuller, T., 2015. Mean of the typical decoding rates: A new translation efficiency index based on the analysis of ribosome profiling data. G3 Genes|Genomes|Genetics 5, 73–80. URL: https://doi.org/10.1534/g3.114.015099, doi:10.1534/g3.114.015099, arXiv:https://academic.oup.com/g3journal/article-pdf/5/1/73/37191150/g3journal0073.pdf.

[9] Devlin, J., Chang, M.W., Lee, K., Toutanova, K., 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. URL: https://arxiv.org/abs/1810.04805, arXiv:1810.04805.

[10] Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., Houlsby, N., 2021. An image is worth 16x16 words: Transformers for image recognition at scale. URL: https://arxiv.org/abs/2010.11929, arXiv:2010.11929.

[11] Fu, H., Liang, Y., Zhong, X., Pan, Z., Huang, L., Zhang, H., Xu, Y., Zhou, W., Liu, Z., 2020. Codon optimization with deep learning to enhance protein expression. Scientific Reports 10, 17617. URL: https://doi.org/10.1038/s41598-020-74091-z, doi:10.1038/s41598-020-74091-z.

[12] GenScript, . Genscript codon frequency table. https://www.genscript.com/tools/codon-frequency-table. Accessed: 2024/07/03.

[13] Gong, H., Wen, J., Luo, R., Feng, Y., Guo, J., Fu, H., Zhou, X., 2023. Integrated mrna sequence optimization using deep learning. Briefings in Bioinformatics 24, bbad001. URL: https://doi.org/10.1093/bib/bbad001, doi:10.1093/bib/bbad001, arXiv:https://academic.oup.com/bib/article-pdf/24/1/bbad001/48782582/bbad001.pdf.

[14] Goulet, D.R., Yan, Y., Agrawal, P., Waight, A.B., Mak, A.N.s., Zhu, Y., 2023. Codon optimization using a recurrent neural network. Journal of Computational Biology 30, 70–81. URL: https://doi.org/10.1089/cmb.2021.0458, doi:10.1089/cmb.2021.0458, arXiv:https://doi.org/10.1089/cmb.2021.0458. pMID: 35727687.

[15] Jain, R., Jain, A., Mauro, E., LeShane, K., Densmore, D., 2023. Icor: improving codon optimization with recurrent neural networks. BMC Bioinformatics 24, 132. URL: https://doi.org/10.1186/s12859-023-05246-8, doi:10.1186/s12859-023-05246-8.

[16] Kingma, D.P., Ba, J., 2017. Adam: A method for stochastic optimization. URL: https://arxiv.org/abs/1412.6980, arXiv:1412.6980.

[17] Kothari, S., Oh, H., 1993. Neural networks for pattern recognition, Elsevier. volume 37 of *Advances in Computers*, pp. 119–166. URL: https://www.sciencedirect.com/science/article/pii/S0065245808604040, doi:https://doi.org/10.1016/S0065-2458(08)60404-0.

[18] Kursuncu, U., Gaur, M., Sheth, A., 2019. Knowledge infused learning (k-il): Towards deep incorporation of knowledge in deep learning. arXiv preprint arXiv:1912.00512 .

[19] Li, S., Moayedpour, S., Li, R., Bailey, M., Riahi, S., Kogler-Anele, L., Miladi, M., Miner, J., Zheng, D., Wang, J., et al., 2023. Codonbert: Large language models for mrna design and optimization. bioRxiv , 2023–09.

[20] Liu, M., Jin, Z., Xiang, Q., He, H., Huang, Y., Long, M., Wu, J., Zhi Huang, C., Mao, C., Zuo, H., 2024a. Rational design of untranslated regions to enhance gene expression. Journal of Molecular Biology 436, 168804. URL: https://www.sciencedirect.com/science/article/pii/S0022283624004261, doi:https://doi.org/10.1016/j.jmb.2024.168804.

[21] Liu, Y., Gao, J., Zhang, X., Fang, X., 2024b. Joint design of 5' untranslated region and coding sequence of mrna. URL: https://arxiv.org/abs/2410.20781, arXiv:2410.20781.

[22] Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S., Guo, B., 2021. Swin transformer: Hierarchical vision transformer using shifted windows. arXiv preprint arXiv:2103.14030 .

[23] Loshchilov, I., Hutter, F., 2019. Decoupled weight decay regularization. URL: https://arxiv.org/abs/1711.05101, arXiv:1711.05101.

[24] Mathews, D.H., Andre, T.C., Kim, J., Turner, D.H., Zuker, M., 1997. An Updated Recursive Algorithm for RNA Secondary Structure Prediction with Improved Thermodynamic Parameters. American Chemical Society. volume 682 of *ACS Symposium Series*. pp. 246–257. URL: https://doi.org/10.1021/bk-1998-0682.ch015, doi:10.1021/bk-1998-0682.ch015. 0.

[25] Mathews, D.H., Disney, M.D., Childs, J.L., Schroeder, S.J., Zuker, M., Turner, D.H., 2004a. Incorporating chemical modification constraints into a dynamic programming algorithm for prediction of rna secondary structure. Proceedings of the National Academy of Sciences 101, 7287–7292. URL: https://www.pnas.org/doi/abs/10.1073/pnas.0401799101, doi:10.1073/pnas.0401799101, arXiv:https://www.pnas.org/doi/pdf/10.1073/pnas.0401799101.

[26] Mathews, D.H., Disney, M.D., Childs, J.L., Schroeder, S.J., Zuker, M., Turner, D.H., 2004b. Incorporating chemical modification constraints into a dynamic programming algorithm for prediction of rna secondary structure. Proceedings of the National Academy of Sciences 101, 7287–7292. URL: https://www.pnas.org/doi/abs/10.1073/pnas.0401799101, doi:10.1073/pnas.0401799101, arXiv:https://www.pnas.org/doi/pdf/10.1073/pnas.0401799101.

[27] Mathews, D.H., Sabina, J., Zuker, M., Turner, D.H., 1999. Expanded sequence dependence of thermodynamic parameters improves prediction of rna secondary structure11edited by i. tinoco. Journal of Molecular Biology 288, 911–940. URL: https://www.sciencedirect.com/science/article/pii/S0022283699927006, doi:https://doi.org/10.1006/jmbi.1999.2700.

[28] Nabeel Asim, M., Imran Malik, M., Dengel, A., Ahmed, S., 2020. K-mer neural embedding performance analysis using amino acid codons, in: 2020 International Joint Conference on Neural Networks (IJCNN), pp. 1–8. doi:10.1109/IJCNN48605.2020.9206892.

[29] National Center for Biotechnology Information (NCBI), 1988–a. Ncbi genome database. URL: https://www.ncbi.nlm.nih.gov/datasets/genome/. [cited 2025 Jan 16].

[30] National Center for Biotechnology Information (NCBI), 1988–b. Ncbi virus database. URL: https://www.ncbi.nlm.nih.gov/labs/virus/vssi/#/. [cited 2025 Jan 16].

[31] Pandya, S., Gadekallu, T.R., Reddy, P.K., Wang, W., Alazab, M., 2022. Infusedheart: A novel knowledge-infused learning framework for diagnosis of cardiovascular events. IEEE Transactions on Computational Social Systems 11, 3060–3069.

[32] Paremskaia, A.I., Kogan, A.A., Murashkina, A., Naumova, D.A., Satish, A., Abramov, I.S., Feoktistova, S.G., Mityaeva, O.N., Deviatkin, A.A., Volchkov, P.Y., 2024. Codon-optimization in gene therapy: promises, prospects and challenges. Front Bioeng Biotechnol 12, 1371596.

[33] Presnyak, V., Alhusaini, N., Chen, Y.H., Martin, S., Morris, N., Kline, N., Olson, S., Weinberg, D., Baker, K.E., Graveley, B.R., Coller, J., 2015. Codon optimality is a major determinant of mRNA stability. Cell 160, 1111–1124.

[34] Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., Liu, P.J., 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. Journal of Machine Learning Research 21, 1–67. URL: http://jmlr.org/papers/v21/20-074.html.

[35] Raissi, M., Perdikaris, P., Karniadakis, G.E., 2019. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. Journal of Computational physics 378, 686–707.

[36] dos Reis, M., Wernisch, L., Savva, R., 2003. Unexpected correlations between gene expression and codon usage bias from microarray data for the whole escherichia coli K-12 genome. Nucleic Acids Res 31, 6976–6985.

[37] Reis, M.d., Savva, R., Wernisch, L., 2004. Solving the riddle of codon usage preferences: a test for translational selection. Nucleic Acids Research 32, 5036–5044. URL: https://doi.org/10.1093/nar/gkh834, doi:10.1093/nar/gkh834, arXiv:https://academic.oup.com/nar/article-pdf/32/17/5036/4156320/gkh834.pdf.

[38] Reuter, J.S., Mathews, D.H., 2010. Rnastructure: software for rna secondary structure prediction and analysis. BMC Bioinformatics 11, 129. URL: https://doi.org/10.1186/1471-2105-11-129, doi:10.1186/1471-2105-11-129.

[39] Sharp, P.M., Li, W.H., 1987. The codon adaptation index-a measure of directional synonymous codon usage bias, and its potential applications. Nucleic Acids Research 15, 1281–1295. URL: https://doi.org/10.1093/nar/15.3.1281, doi:10.1093/nar/15.3.1281, arXiv:https://academic.oup.com/nar/article-pdf/15/3/1281/3988268/15-3-1281.pdf.

[40] Taneda, A., Asai, K., 2020. Cosmo: A dynamic programming algorithm for multicriteria codon optimization. Computational and Structural Biotechnology Journal 18, 1811–1818. URL: https://www.sciencedirect.com/science/article/pii/S200103702030324X, doi:https://doi.org/10.1016/j.csbj.2020.06.035.

[41] Terai, G., Kamegai, S., Asai, K., 2016. Cdsfold: an algorithm for designing a protein-coding sequence with the most stable secondary structure. Bioinformatics (Oxford, England) 32, 828—834. URL: https://doi.org/10.1093/bioinformatics/btv678, doi:10.1093/bioinformatics/btv678.

[42] Tolstikhin, I.O., Houlsby, N., Kolesnikov, A., Beyer, L., Zhai, X., Unterthiner, T., Yung, J., Steiner, A., Keysers, D., Uszkoreit, J., Lucic, M., Dosovitskiy, A., 2021. Mlp-mixer: An all-mlp architecture for vision, in: Ranzato, M., Beygelzimer, A., Dauphin, Y., Liang, P., Vaughan, J.W. (Eds.), Advances in Neural Information Processing Systems, Curran Associates, Inc.. pp. 24261–24272.

[43] Vostrosablin, N., Lim, S., Gopal, P., Brazdilova, K., Parajuli, S., Wei, X., Gromek, A., Prihoda, D., Spale, M., Muzdalo, A., Greig, J., Yeo, C., Wardyn, J., Mejzlik, P., Henry, B., Partridge, A.W., Bitton, D.A., 2024. mrnaid, an open-source platform for therapeutic mrna design and optimization strategies. NAR Genomics and Bioinformatics 6, lqae028. URL: https://doi.org/10.1093/nargab/lqae028, doi:10.1093/nargab/lqae028, arXiv:https://academic.oup.com/nargab/article-pdf/6/1/lqae028/56958324/lqae028.pdf.

[44] Zhang, H., Liu, H., Xu, Y., Huang, H., Liu, Y., Wang, J., Qin, Y., Wang, H., Ma, L., Xun, Z., Hou, X., Lu, T.K., Cao, J., . Deep generative models design mrna sequences with enhanced translational capacity and stability. Science 0, eadr8470. URL: https://www.science.org/doi/abs/10.1126/science.adr8470, doi:10.1126/science.adr8470, arXiv:https://www.science.org/doi/pdf/10.1126/science.adr8470.

[45] Zhang, H., Zhang, L., Lin, A., Xu, C., Li, Z., Liu, K., Liu, B., Ma, X., Zhao, F., Jiang, H., Chen, C., Shen, H., Li, H., Mathews, D.H., Zhang, Y., Huang, L., 2023. Algorithm for optimized mrna design improves stability and immunogenicity. Nature 621, 396–403. URL: https://doi.org/10.1038/s41586-023-06127-z, doi:10.1038/s41586-023-06127-z.

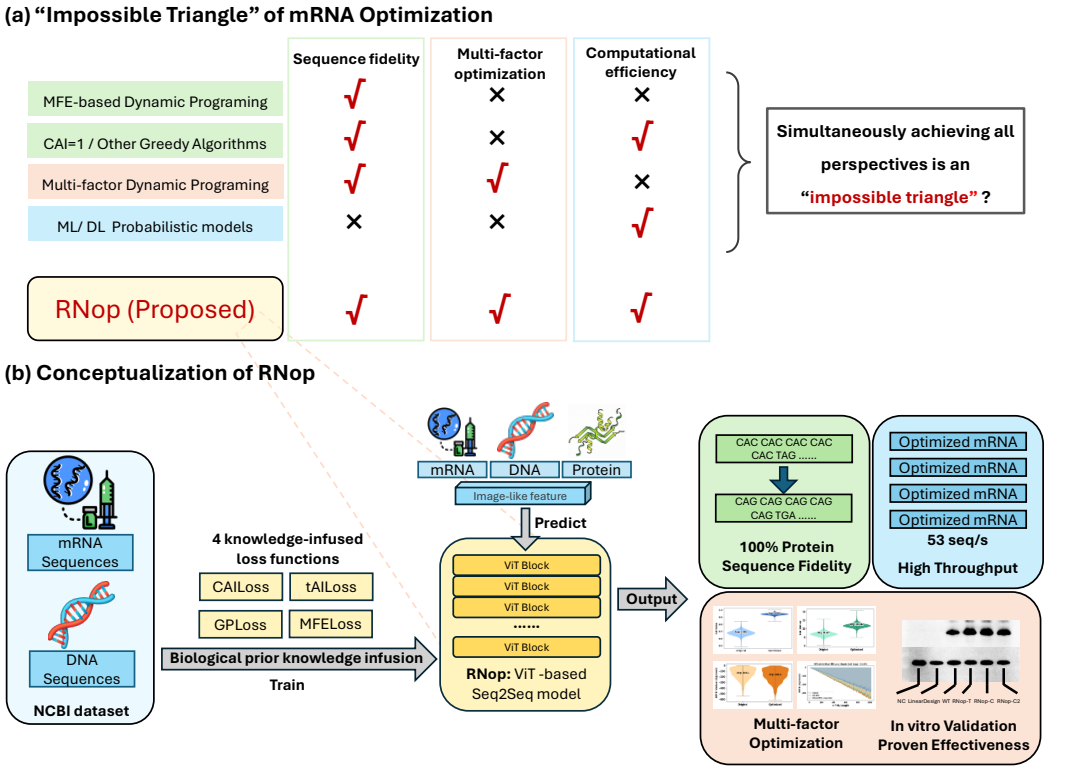


Figure 1: (a) **The "impossible triangle" of mRNA optimization.** Simultaneously achieving high sequence fidelity, computational efficiency, and a wide range of optimization objectives remains a significant challenge for current mRNA design methods. We propose a new knowledge-infused deep learning method to achieve all factors concurrently. (c) **The conceptualization of RNop**: Our methods include 4 knowledge-infused loss functions leveraging biological prior knowledge. RNop can process various sequences through a ViT-based Sequence-to-Sequence architecture. The optimized sequences exhibit great expression improvements in both computerized and biological experiments. (c) **Effect of infused Knowledge**. After the input process, each codon has a probability space spanning the whole vocabulary. The infused knowledge helps RNop models reduce the information entropy, providing the preferred codon for the input codon in specific species. The output is the optimal, synonymous codon.

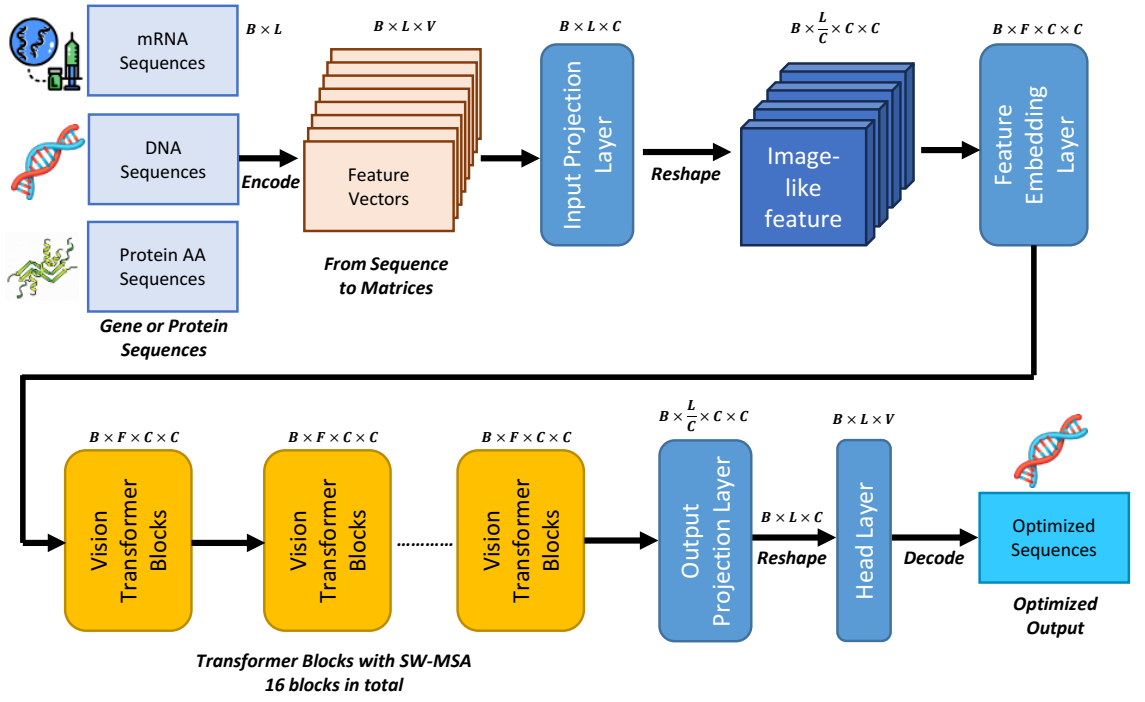


Figure 2: The architecture of our RNop model: (1) RNop model uses our coding method to encode multiple kinds of gene sequences to feature matrices. (2) The feature matrices are processed and reshaped to image-like features. (3) Then, the model can process these features using Vision Transformer structures. (4) The features are unpacked and restored to sequences and decoded to the desired optimal sequences.

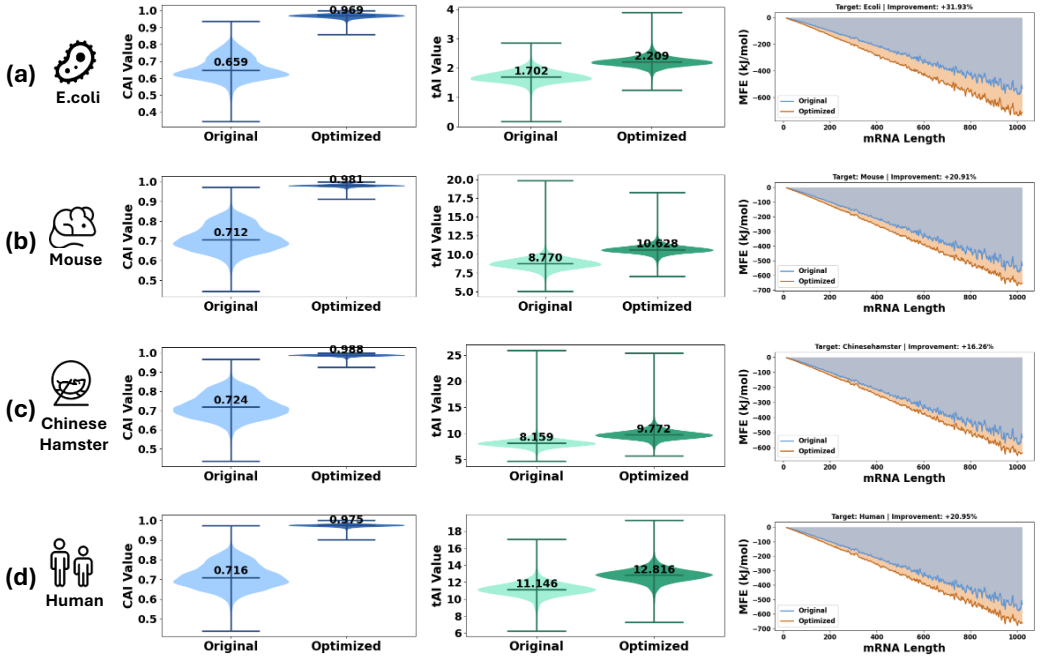


Figure 3: The histogram of overall CAI, tAI, and MFE comparison between original and optimized mRNA sequences. We applied the RNop model to the test dataset with different target species ((a) E. coli, (b) mouse, (c) Chinese hamster, (d) human). No sequence length or codon error was observed. The optimized mRNA sequences have higher overall CAI, tAI, and MFE values, which indicate better codon adaptiveness, tRNA affinity, and chemical stability in the target species.

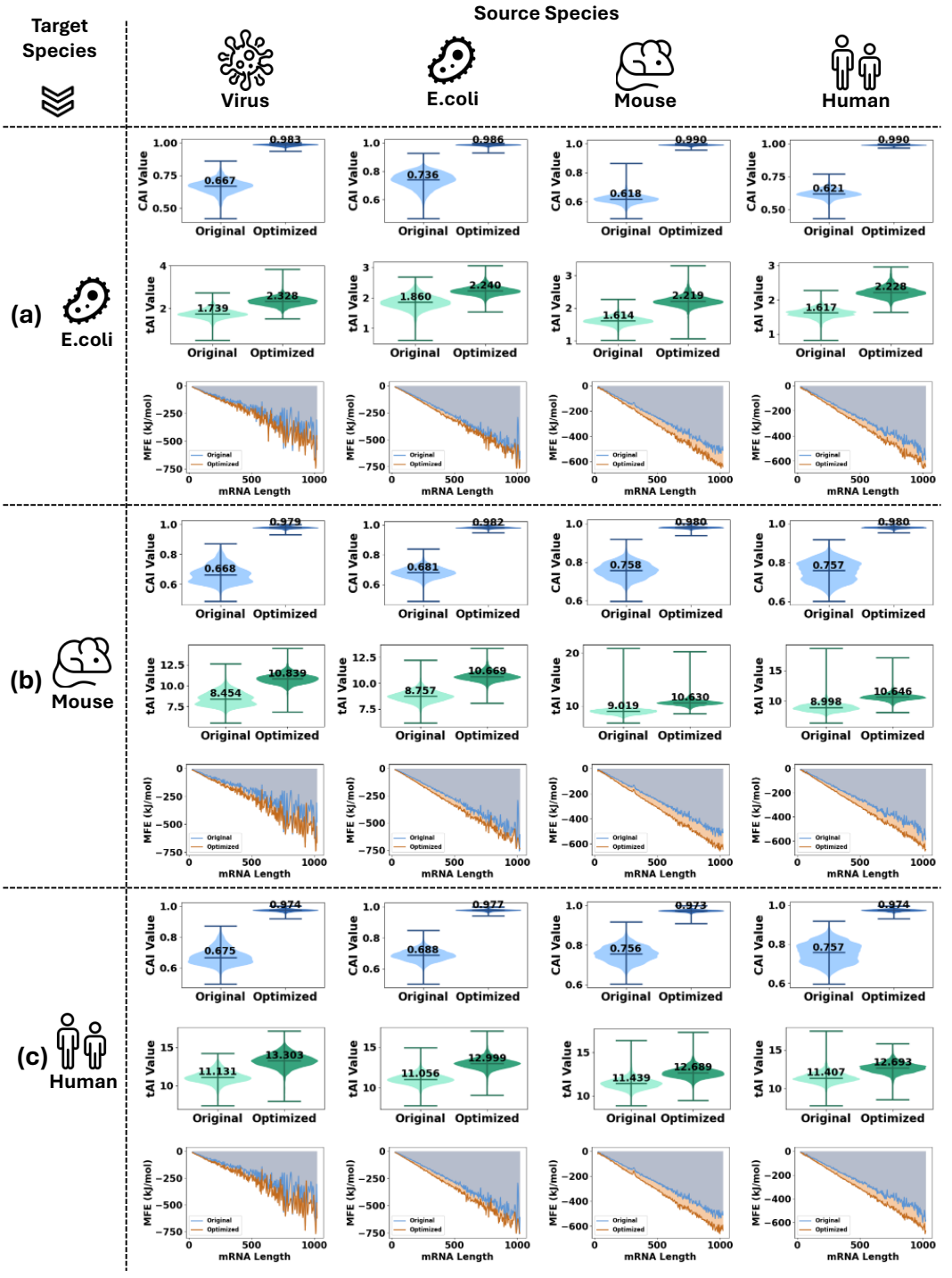


Figure 4: The histogram of CAI, tAI, and MFE comparison between mRNA sequences from different species No sequence length or codon error was observed. (a) The optimized mRNA sequences have better overall CAI, tAI, and MFE values. The data resources are virus, E. coli, mouse, and human, respectively. And the target species is "E. coli". Both the heterologous and homologous sequences are optimized. (b) The target species is "Mouse". (c) The target species is "Human".

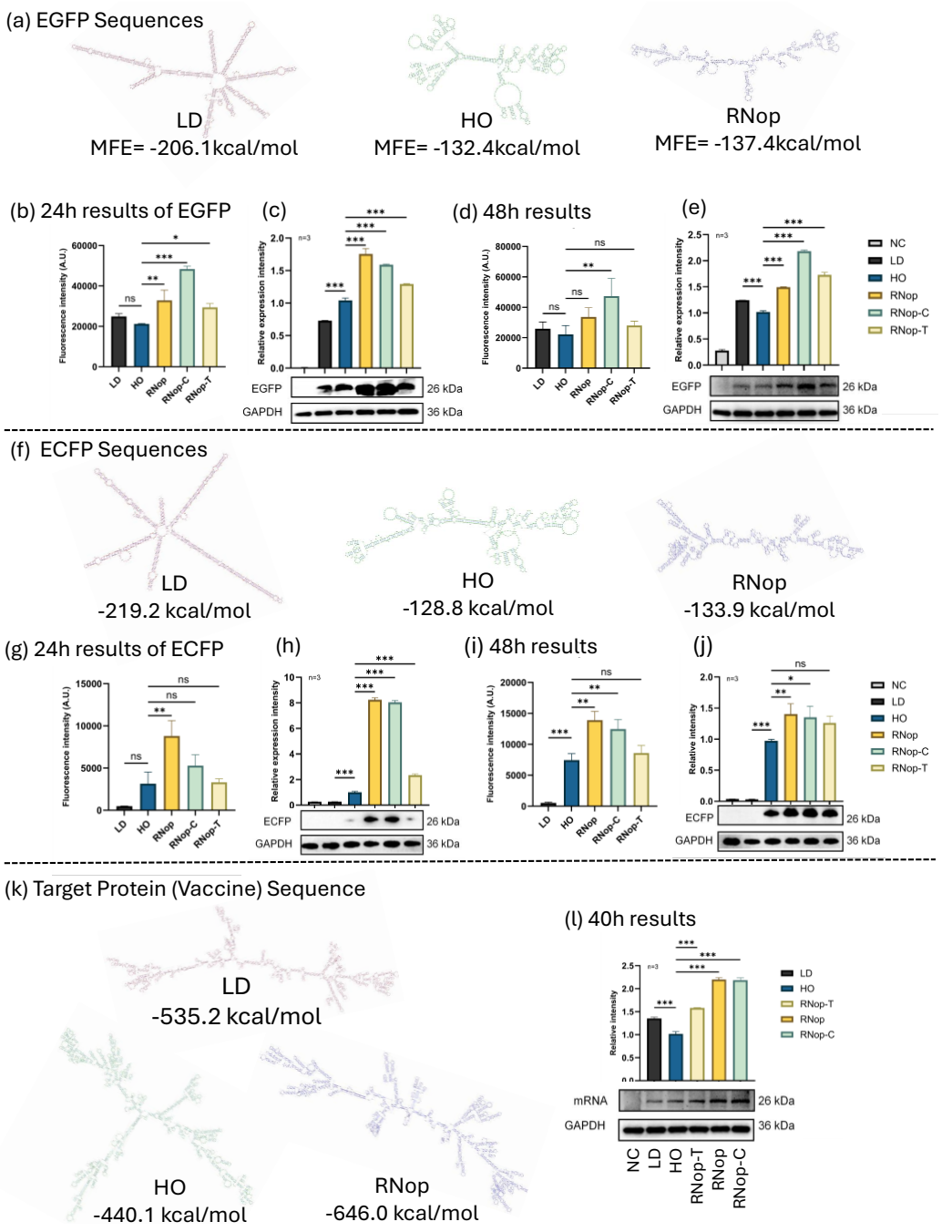


Figure 5: *In vitro* analysis of mRNA sequence optimization by the RNop model. We applied the fluorescent enzyme-labeled instrument and western blot to measure the expression rate of mRNA sequences and summarized these results through densitometric analysis. The Glyceraldehyde-3-phosphate Dehydrogenase (labeled "GAPDH" in the chart) was selected as trimimg standard for cell amounts. (a) The visualization of optimized EGFP sequences. Fluorescent and western blot results of EGFP expression at 24 h and 48 h after mRNA transfection are summarized in (b), (d), and (c), (e); (f) The visualization of ECFP. Results at 24 h and 48 h after mRNA transfection are summarized in (g), (i), and (h), (j); The visualization (k) and expression results (l) of the target protein. Replications $n = 3, * p < 0.05, ** p < 0.01, *** p < 0.001$.

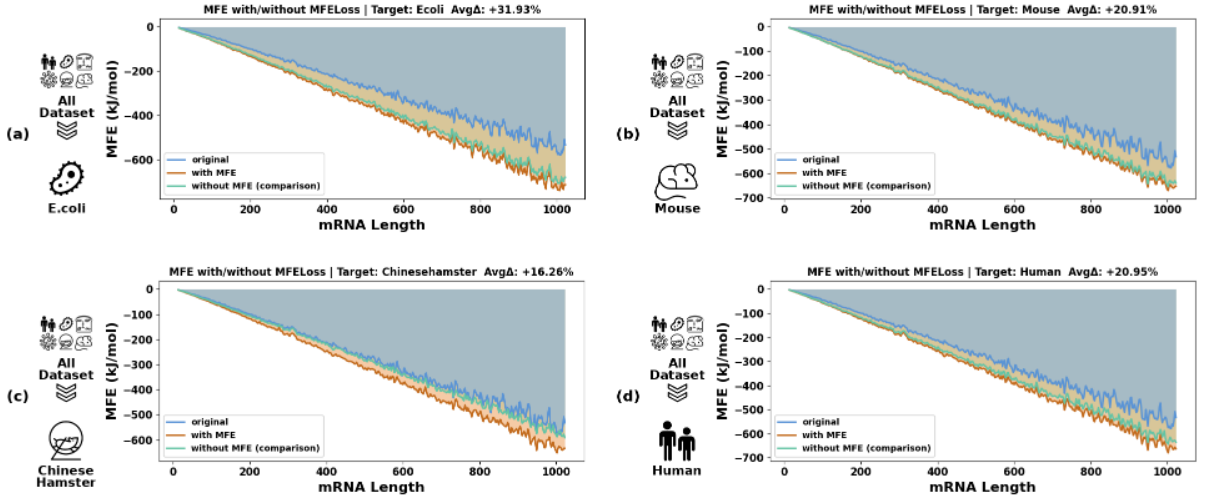


Figure 6: The histogram of MFE comparison between sequences original, optimized, and optimized without MFELoss. (a)Results of setting target species "E. Coli"; (b)Results of setting target species "Mouse"; (c) Results of setting target species "Chinese hamster"; (d)Results of setting target species "Human". No incorrect sequence length or codon error was observed. The optimized mRNA sequences have lower MFE values than the original sequences. Meanwhile, they also surpass the optimization results from the RNop model with the MFELoss disabled. These experiments demonstrate that the MFELoss works effectively in improving the MFE of mRNA sequences.

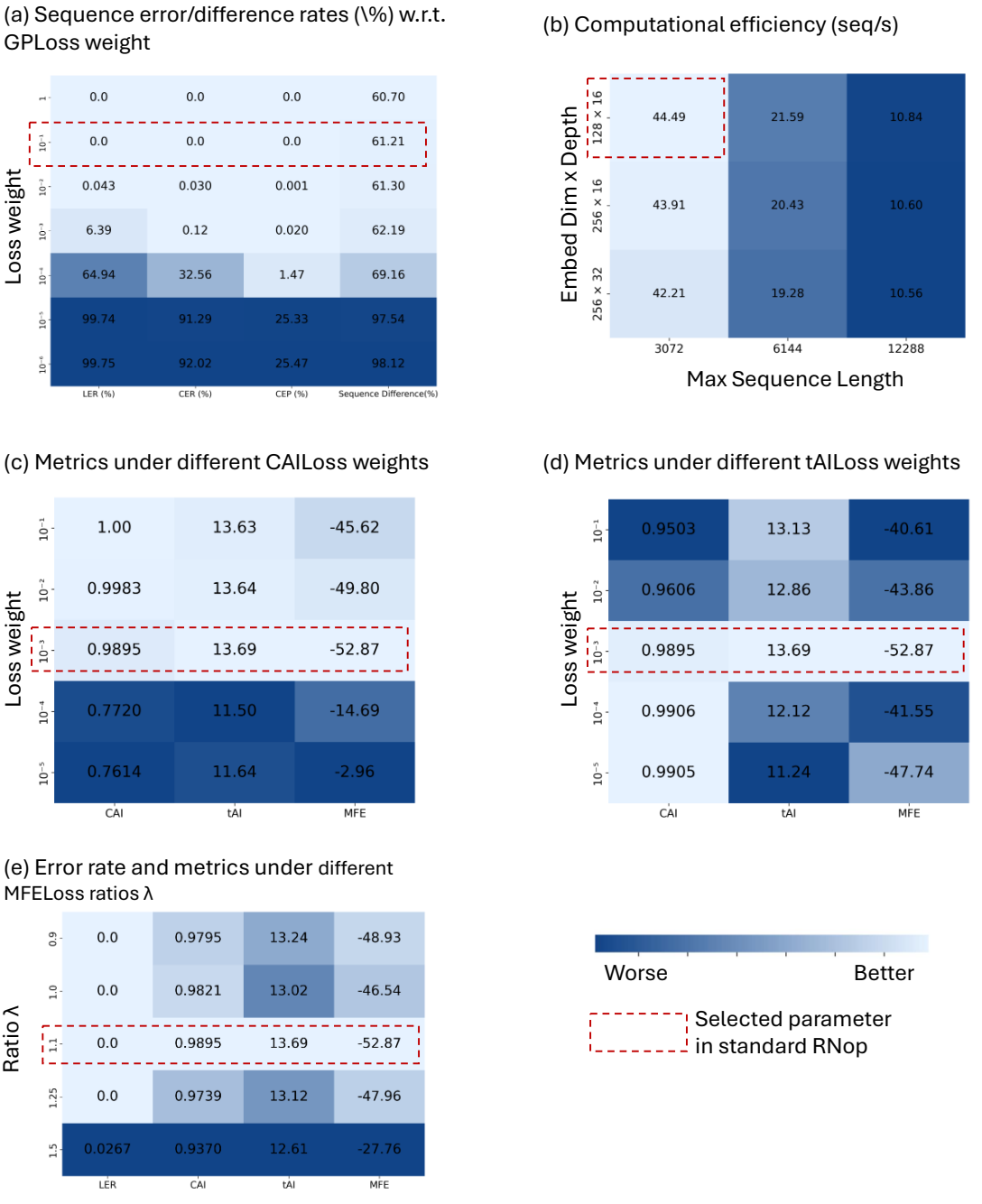


Figure 7: The results of throughput and interpretability. (a) Results of sequence error/difference rates (%) w.r.t. GPLoss weight, with setting target species "Human". 10^{-1} is enough for synonymous retention; (b) Computational efficiency (sequences per second) under different combinations of embedding dimension, depth of RNop, and max sequence length; (c) CAI, tAI, MFE under different CAILoss weights. Properly setting the loss weight can obtain sequences with best overall metrics; (d)CAI, tAI, MFE under different tAILoss weights; (e)LER, CAI, tAI, MFE under different MFELoss ratios λ. Setting λ too high resulted in unconvergence, while setting it too low resulted in ignoring the MFE optimization.